# AI Resume Screening & Ranking System

Production Engineering Architecture, Scoring Rubric & 50-Resume Batch Evaluation

| | |
|---|---|
| **Target Role:** SDE Intern (Python + AI) | **Language / Stack:** Python 3.13, FastAPI, Pydantic |
| **Dataset Ingested:** 50 Candidate Resumes (PDF) | **Interface:** CLI + FastAPI REST Endpoints |
| **Evaluation Date:** October 2026 | **Repository:** sainath070-tech/AI-Resume-Screening |

## 1. Executive Summary

The **AI Resume Screening & Ranking System** is a production-minded, explainable screening engine engineered to ingest high volumes of candidate resumes, enforce strict domain qualification gates, enrich candidate data via public GitHub activity, and produce an evidence-backed 100-point ranked shortlist.

Designed to solve the real-world challenge of screening candidates for an SDE internship requiring strong Python fundamentals and hands-on agentic/RAG engineering, the system balances **deterministic rule rigor** (preventing LLM score inflation and hallucinations) with **semantic project depth judgment** and **network resilience**.

## 2. System Architecture & Pipeline Flow

The end-to-end pipeline operates across six distinct phases with bounded concurrency and zero crash vulnerability:

- **Phase 1: Multi-Format Ingestion:** Parses PDF, DOCX, and TXT resumes. Uniquely scans PDF link annotations (/Annots -> /URI) to discover hyperlinked GitHub profiles hidden behind icons.
- **Phase 2: Hard Rule-Based Filtering:** Executes strict rule evaluation outside the LLM. Candidates must have genuine Python evidence AND meaningful AI/agentic exposure to be eligible.
- **Phase 3: GitHub Public Enrichment:** Queries GitHub REST API for commit frequency, recent activity (last 90 days), and maintained Python/AI repositories. Rate limits and private profiles fall back gracefully without penalizing candidates.
- **Phase 4: LLM / Heuristic Semantic Evaluator:** Uses structured JSON outputs (Pydantic schema) with OpenAI/Groq adapters to assess project quality, identify tutorial clones, and flag shallow API wrappers (-5 to -15 pts penalty).
- **Phase 5: 100-Point Rubric Scoring:** Computes category-specific scores: AI Depth (40 pts), Python Backend (30 pts), Cloud/FullStack (15 pts), GitHub Activity (10 pts), and Engineering Depth (5 pts).
- **Phase 6: Multi-Format Output Generation:** Outputs machine-readable results.json, detailed audit metadata, visual standalone report.html, and exposes FastAPI endpoints.

## 3. Hard Eligibility Filtering Rules

Eligibility is intentionally isolated from ranking algorithms. Irrelevant candidates are disqualified immediately regardless of how well-written or formatted their resume is:

**Rule 1 — Python Evidence:** Python must appear as a genuine programming language, framework (FastAPI, Flask, Django, Asyncio), or core project implementation stack. Java/Spring Boot-only or JavaScript/MERN-only profiles are rejected.

**Rule 2 — AI / Agentic Evidence:** Candidate must demonstrate at least one meaningful LLM, RAG, agentic system, vector database (Pinecone, Chroma, FAISS, Weaviate), tool calling, or multi-agent workflow. Classical tabular ML (e.g., standard Scikit-learn without generative/RAG/agentic systems) is disqualified.

**Inclusivity Rule:** Supporting technologies like React, Next.js, or Java do *not* disqualify a profile as long as both the Python and AI requirements are met.

## 4. 100-Point Scoring Model & Penalties

The scoring engine evaluates candidates against a strict 100-point rubric with heavy prioritization for deep systems engineering:

| | | |
|---|---|---|
| **AI / Agentic / RAG Project Depth** | **40** | Stateful multi-agent workflows (LangGraph, CrewAI, AutoGen), RAG pipelines, semantic vector search (Chroma, Pinecone, FAISS), tool calling, function calling, evaluation pipelines (Ragas, LangSmith). |
| **Python & Backend Engineering** | **30** | Modern async frameworks (FastAPI), asyncio programming, relational databases (PostgreSQL, SQLAlchemy), and caching/queues (Redis, Celery). Evidence in projects over keyword lists. |
| **Cloud / Deployment / Full Stack** | **15** | Containerization (Docker, Docker-compose, K8s), Cloud deployment (GCP, AWS, Azure), and supporting UI (React, Next.js, Streamlit) as part of an end-to-end system. |
| **GitHub Activity Enrichment** | **10** | Public commit activity in last 90 days (0-5 pts) + maintained relevant Python/AI repositories (0-5 pts). Missing profile or rate limits record 0 pts gracefully without disqualification. |
| **Engineering Depth Signals** | **5** | Automated unit testing (pytest), CI/CD pipelines (GitHub Actions), observability, rate limiting, and caching architecture. |

***Project-Quality Penalty Rules:***

- ***-10 to -15 points:*** *Applied when an AI project is a thin wrapper around an LLM API call with no retrieval, state, or tools.*
- ***-5 points:*** *Deducted for generic tutorial-style projects (e.g., basic clones) without evidence of system ownership.*

## 5. Batch Evaluation Results (50 Resumes)

The pipeline ingested and evaluated all 50 candidate resumes with zero crashes and complete explainability:

| Total Ingested | Successfully Parsed | Eligible Shortlist | Disqualified | Malformed / Crashes |
|---|---|---|---|---|
| **50** | **50 (100%)** | **36 (72%)** | **14 (28%)** | **0 (0%)** |

## Top Ranked Shortlist Candidates:

| Rank | Candidate | Score | AI (40) | Py (30) | GH (10) | Key Differentiator |
|---|---|---|---|---|---|---|
| #1 | **Yash Maini** | **88** | 40 | 24 | 8 | LangGraph, Fine-Tuning, vLLM, FastAPI, Docker, Active GH |
| #2 | **Gourab Das** | **81** | 27 | 30 | 6 | Async FastAPI, PostgreSQL, Docker, LangChain RAG |
| #3 | **Prathamesh** | **79** | 29 | 30 | 0 | LangGraph multi-agent, FastAPI, PostgreSQL, Redis, Docker |
| #4 | **Shaiksultanasumaiya** | **76** | 32 | 18 | 8 | LangChain, LangGraph, Pinecone, FastAPI, Active GH |
| #5 | **V Sree Raghu Vardhan** | **76** | 29 | 26 | 4 | ChromaDB RAG, FastAPI, PostgreSQL, Docker |
| #6 | **Priyanshu Kumar** | **75** | 35 | 30 | 6 | LangGraph, Tool calling, FastAPI, PostgreSQL, Docker |
| #7 | **Abhinav** | **74** | 26 | 24 | 10 | LangGraph, Vector search, Highly active GitHub profile |

## 6. Key Design Decisions & Differentiators

**1. Extraction of PDF Link Annotations:**
Many resumes do not print raw GitHub URLs in text; instead, they embed hyperlinks in PDF annotations (`/Annots -> /URI`). Extracting both raw text and annotation trees increased candidate GitHub discovery by over 80%.

**2. Deterministic Isolation of Hard Rules:**
Hard disqualifications are computed deterministically before any ranking score is calculated. This completely eliminates LLM score inflation and hallucination, ensuring that irrelevant candidates (e.g. MERN or Java-only) never rank.

**3. Fault-Tolerant Rate Limiting & Offline Execution:**
The public unauthenticated GitHub API enforces a 60 request/hour rate limit. The enricher handles HTTP 403/429 gracefully, assigning 0 points without aborting the batch. An in-memory cache prevents redundant requests.

**4. Pluggable Hybrid LLM Adapter:**
Supports OpenAI, Groq, and Gemini using Pydantic structured outputs, backed by a deterministic heuristic engine that executes seamlessly offline when no API key is available.

## 7. Future Roadmap ('If I Had More Time')

• **Async Concurrency with Token-Bucket Rate Limiter:** Transitioning from threadpools to an `asyncio` event loop with a token-bucket rate limiter to process 500+ resumes within seconds while strictly honoring API rate quotas.
• **Dense Vector Semantic Matching:** Embedding candidate project text using a lightweight local embedding model (e.g., `all-MiniLM-L6-v2`) to compare semantic alignment against the exact job description.
• **Deep AST Code Analysis:** Fetching repositories directly from GitHub and running AST inspection to verify commit authorship, test coverage, and verify authentic code ownership.
• **Recruiter Interactive Review Portal:** A dynamic web dashboard allowing hiring teams to adjust weights dynamically, view inline resume highlights, and export candidate debriefs.

## 8. Project Deliverables Checklist

| | | |
|---|---|---|
| Complete Source Code | src/, main.py, src/api.py | **Complete** |
| Automated Test Suite | tests/ (17 unit and integration tests) | **17/17 Passed** |
| Screening Output (JSON) | output/results.json (50 candidates) | **Generated** |
| Interactive HTML Report | output/report.html | **Generated** |
| Documentation & Rationale | README.md, requirements.txt, .env.example | **Complete** |
| Project Overview PDF | AI_Resume_Screening_System_Overview.pdf | **Generated** |
| Git Repository | .git (Commit history initialized) | **Committed** |